

전국 자연재해 EDA 프로젝트

(2019~2023)

작성자 : 김태훈

1. 프로젝트 개요

1. 데이터 출처: 행정안전부 공개 통계 (2019~2023)

2. 사용 기술:

가. Python (Pandas, Matplotlib)

나. SQLite

다. ChatGPT

3. 분석 목적:

가. 재난 유형별 피해 규모 파악

나. 연도별 피해 트렌드 분석

다. 정책 수립에 활용 가능한 기반 데이터 도출

2. 데이터 소개 및 전처리 과정

1. 전처리 주요 내용:

가. NaN 포함된 칼럼 제거

나. '구분', '지역명' 컬럼에 대해 ffill() 처리

```
df['분류1'] = df['구분/지역명 <▷>'].ffill()  
df['분류2'] = df['Unnamed: 1'].fillna('')  
df['피해항목'] = df['분류1'] + '_' + df['분류2']
```

다. 2019~2023년 컬럼을 melt() 하여 연도별 데이터로 변환

```
df_melted = pd.melt(df,  
                    id_vars='피해항목',  
                    value_vars=['2019년', '2020년', '2021년', '2022년', '2023년'],  
                    var_name='연도',  
                    value_name='피해규모')
```

3. 미션1 - 전국 피해 Top 5

1. SQL로 피해항목별 총합 정리

```
SELECT 피해항목, SUM(피해금액) AS 총피해  
FROM disaster_data  
GROUP BY 피해항목  
ORDER BY 총피해 DESC  
LIMIT 5;
```

2. SQL로 정리한 내용에서 상위 5개 도출

피해 항목	총피해 (원)
총피해액	2,920,682,118
공공시설_피해액소계	2,438,188,107
공공시설	2,437,989,215
사유시설_피해액소계	184,387,880
건물_피해액	150,502,519

4. 미션2 - 연도별 피해 추이

1. 연도별 피해 총액 집계 (Pandas groupby)

```
year_data = df_melted.groupby('연도')['피해규모'].sum().reset_index()
year_data['피해규모'] = year_data['피해규모'].astype(float)
print("연도별 피해 규모 합계:")
print(year_data.to_string(formatters={'피해규모': '{:,.0f}'.format})))
```

2. 가장 피해가 큰 해: 2020년 (약 3,869억 원)

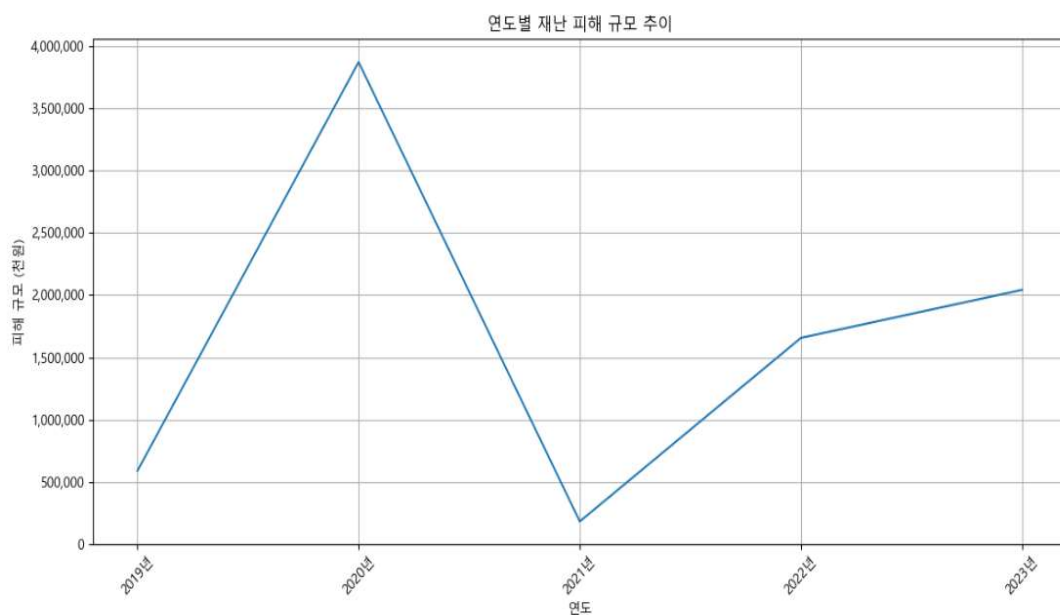
연도	피해총액(원)
2020년	3,869,300,258
2023년	2,041,853,560
2022년	1,656,536,990
2019년	589,553,631
2021년	183,598,841

3. 선 그래프 + 연도별 수치 표시 (시각화) 도출

가. 그래프 도출 작업

```
# 연도별 추이 시각화 (라인차트)
# 한글폰트 적용
plt.rc('font', family='Malgun Gothic')
plt.figure(figsize=(12, 6))
# 피해규모 나누기
year_data['피해(천원)'] = year_data['피해규모'] / 1000 # 천원 단위로 변환
# 그래프 그리기
plt.plot(year_data['연도'], year_data['피해(천원)'])
plt.title('연도별 재난 피해 규모 추이')
plt.xlabel('연도')
plt.ylabel('피해 규모 (천원)')
plt.xticks(rotation=45)
# y축 천 단위 콤마 포맷
ax = plt.gca()
ax.yaxis.set_major_formatter(ticker.FuncFormatter(lambda x, _: f'{int(x):,}'))
plt.grid()
plt.tight_layout()
plt.show()
# 가장 피해가 컸던 해 찾기
max_year = year_data.loc[year_data['피해규모'].idxmax()]
print(f"가장 피해가 컸던 해: {max_year['연도']}년, 피해 규모: {max_year['피해규모']:,.0f}원")
```

나. 그래프 도출



5. 미션3 - 피해원인 및 유형분석

1. 피해항목 -> '재난유형' + '구분' 으로 분리

```
df_melted[['재난유형', '구분']] = df_melted['피해항목'].str.split('_', expand=True)
```

라.

2. 숫자 정제 및 NaN 제거

```
# 피해규모 숫자 타입 확인 및 정제
df_melted['피해규모'] = pd.to_numeric(df_melted['피해규모'], errors='coerce')
# NaN 값 제거
df_melted = df_melted.dropna(subset=['피해규모'])
print("피해규모 데이터 타입 확인:")
print(df_melted['피해규모'].dtype)

# 그룹화해서 재난 유형별 피해 총합 구하고 내림차순 정렬
df_by_type = df_melted.groupby('구분')['피해규모'].sum().reset_index()
df_by_type = df_by_type.sort_values(by='피해규모', ascending=False)
df_by_type['구분'] = df_by_type['구분'].replace('', '총합계')

print("재난 유형별 피해 규모 총합:")
print(df_by_type.to_string(formatters={'피해규모': '{:,.0f}'.format}))
# 합계 계열 제거
drop_list = ['계', '피해액', '피해액소계', '총합계', '기타']
# 해당 값들 제외한 데이터 프레임 재정의
df_by_type = df_by_type[~df_by_type['구분'].isin(drop_list)]
# 피해 규모 내림차순 정렬
df_by_type = df_by_type.sort_values(by='피해규모', ascending=False)
print((df_by_type.to_string(formatters={'피해규모': '{:,.0f}'.format})))
```

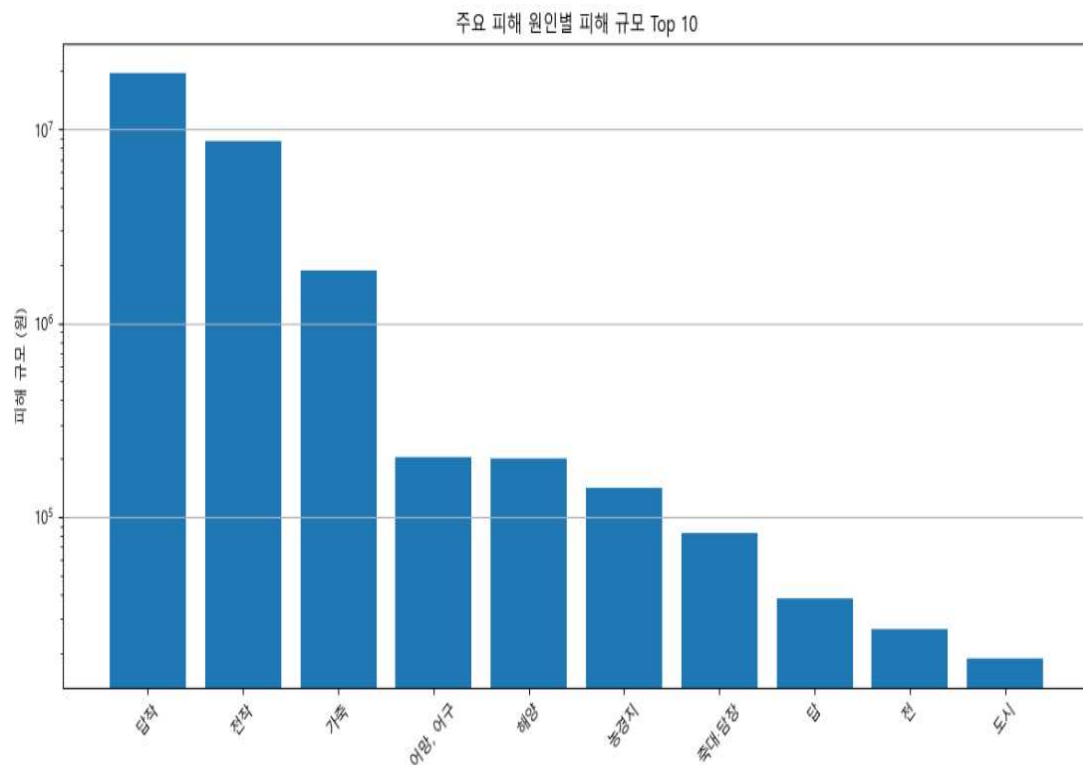
3. 로그 스케일로 피해 유형 Top 10 도출 (시각화)

가. 그래프 도출 작업

```
# 재난 유형별 피해 규모 시각화(상위 10개)
top10 = df_by_type.head(10)

plt.figure(figsize=(12, 6))
plt.bar(top10['구분'], top10['피해규모'])
plt.yscale('log') # 로그 스케일로 Y축 설정
plt.xticks(rotation=45)
plt.title('주요 피해 원인별 피해 규모 Top 10')
plt.ylabel('피해 규모 (원)')
plt.tight_layout()
plt.grid(axis='y')
plt.show()
```

나. 그래프 도출



6. 미션4 - 재난 유형 보고서

1. 재난유형별 + 연도별 피해 트렌드 분석

```
df_trend = df_melted.groupby(['재난유형', '연도'], as_index=False)['피해규모'].sum()
```

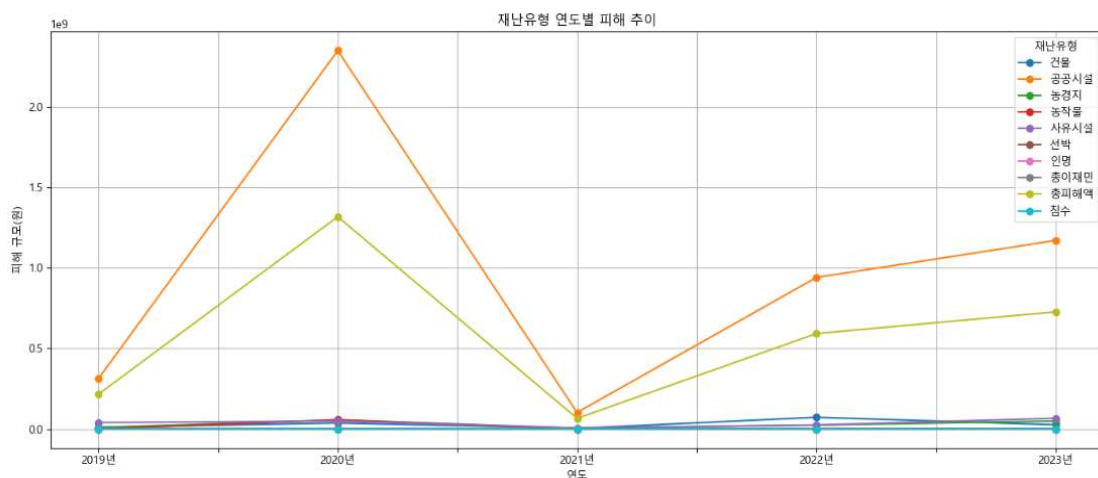
2. 연도별 다중 선 그래프 도출

가. 그래프 도출 작업

```
# 재난유형별로 나누어서 선 그래프로 그리기

pivot = df_trend.pivot(index='연도', columns='재난유형', values='피해규모')
pivot.plot(figsize=(14,6), marker='o')
plt.title("재난유형 연도별 피해 추이")
plt.ylabel("피해 규모(원)")
plt.grid(True)
plt.legend(title='재난유형')
plt.tight_layout()
plt.show()
```

나. 그래프 도출



7. 인사이트 & 해석

1. 2020년은 전체 피해 중 가장 높음
2. 공공시설, 건물, 농경지 피해가 큰 비중 차지
3. 해마다 반복적으로 집중되는 유형 있음
(정책 설계 필요성)

8. 플랫폼 기술 및 모델 구조

1. Pandas: melt, groupby, str.split, to_numeric
2. Matplotlib: 로그스케일, 라벨, 포맷터 설정 등
3. SQLite + SQL: 로컬 저장 및 쿼리 조회
4. Chat GPT: 흐름 정리 및 코드 구조 도움

흐름도

Excel -> Pandas 전처리 -> SQLite 저장 -> SQL 조회
-> 시각화 & 인사이트 도출

9. 가치 및 회고

1. ChatGPT와 GitHub를 활용한 첫 개인 데이터 분석

프로젝트였습니다.

데이터 수집, 전처리, 시각화를 통해 연도별
재난 피해 트렌드를 분석하고, 혼자서도 데이터
프로젝트를 구성해낼 수 있다는 자신감을 얻었습니다.